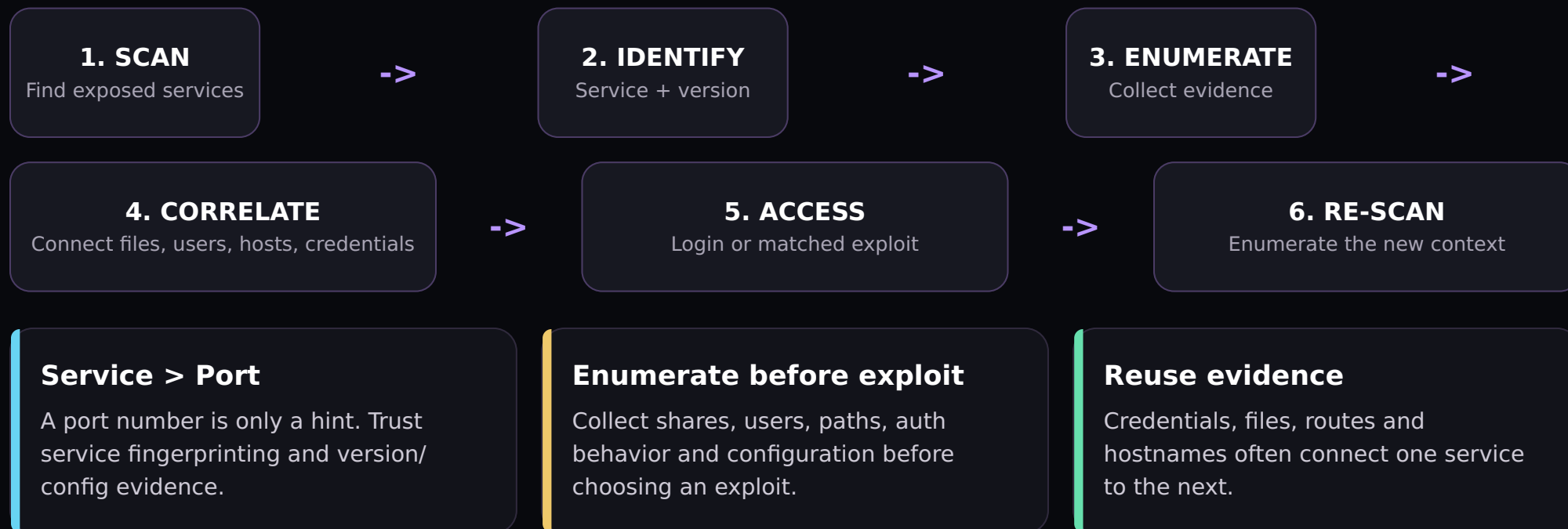
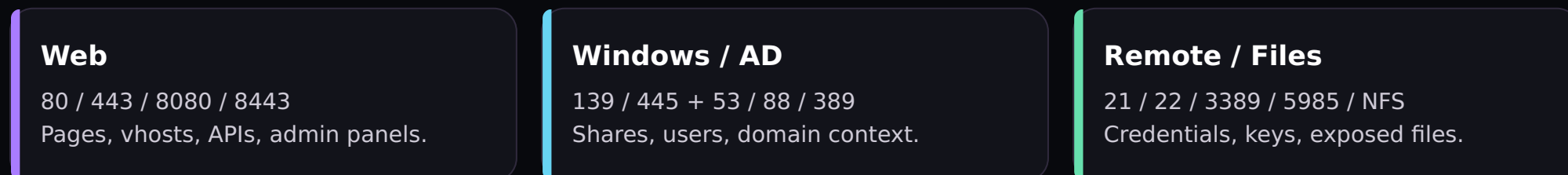


Master Port Decision Map

A visual routing reference: scan -> identify -> enumerate -> correlate -> access -> re-enumerate.



Open the branch that matches the service cluster



One-page decision tree

Follow the service and the evidence - not a memorized exploit.

TARGET

Discover ports

`nmap -Pn -p- TARGET`

Detail what is open

`nmap -sC -sV -p PORTS TARGET`

Write one question

What do I need to learn next?

Web surface

Tech -> paths/vhosts -> auth/inputs -> relevant test.

Start: Browser, curl, Burp, Gobuster/ffuf.

Windows / AD

Shares/users/signing/domain -> files or credentials -> validated access.

Start: smbclient, enum4linux-ng, NetExec.

Remote access

Banner/auth state -> known credential/key -> session.

SSH, FTP, RDP, WinRM, VNC.

Network information

Records/community/exports -> names, users, routes, new targets.

dig, snmpwalk, rpcinfo, showmount.

Data services

Auth/config/databases -> map findings back to the application.

mysql, psql, redis-cli, curl.

Specialized service

Fingerprint product/version/config first, then research a precise match.

Docker, AJP, RMI, Rsync, MQ, search.

Every branch returns to the same loop: finding -> login/exploit -> foothold -> re-enumerate -> new service or route.

Group ports by what they usually mean

Use these as routing hints. If a service is on a non-standard port, the fingerprint wins.

Web Surface

80, 443, 8000, 8080, 8443, 8888

Open every web port. Compare apps, vhosts, auth, APIs and admin panels.

Windows / AD

135, 139, 445, 3389, 5985/5986

If 53, 88, 389, 464, 636 or 3268 also appear, think domain context.

Linux / Remote

21 FTP, 22 SSH, 23 Telnet, 873 Rsync

Look for anonymous access, exposed files, keys and verified credentials.

Network Info

53 DNS, 111 RPC, 161 SNMP, 2049 NFS

Often reveals hostnames, users, exports, interfaces, routes and new targets.

Mail / Identity

25, 110, 143, 88, 389/636

Banners, naming context and known credentials can reveal user/domain structure.

Databases

1433, 3306, 5432, 6379, 27017

Check auth/config, then map connection strings back to web or application files.

Rule of thumb: Port -> Service -> Version/Config -> Enumeration -> Finding -> Access. Never jump directly from a port number to an exploit.

Web surface decision map

Treat each HTTP/HTTPS-like port as a separate application until proven otherwise.

1	Open + fingerprint Browser, headers, TLS, technology.	Question What app/framework is actually here?	Start curl -I, browser, WhatWeb.
2	Discover content Directories, files, vhosts, alternate routes.	Question Is there another surface or hidden function?	Start Gobuster / ffuf.
3	Understand input Auth, roles, parameters, uploads, API, sessions.	Question What does each input control?	Start Burp Proxy / Repeater.
4	Test the relevant class Only after the behavior is understood.	Question Which vulnerability class fits the evidence?	Examples SQLi, XSS, file handling, auth flaws.
5	Correlate Config -> DB; hostname -> vhost; files -> creds.	Question What new service does this finding unlock?	Return Validate -> access -> re-enumerate.

Open every web port

Do not automate everything at once

Treat vhosts as separate apps

SMB and domain-context map

Decide first whether you are seeing a standalone Windows host or a broader domain context.

If 139 / 445 is open

SMB

ASK

Anonymous access? Shares? Users/domain? Signing? OS/version?

START

smbclient -L, enum4linux-ng, NetExec.

Useful evidence

Readable share -> config/backup/key -> credential or hostname -> validate against a service that actually exists.

Strong domain-controller clue

Look for the cluster

53 DNS · 88 Kerberos · 135 RPC · 139 SMB · 389 LDAP · 445 SMB · 464 kpasswd · 636 LDAPS · 3268 GC

Then ask

What is the domain/naming context? Which users are known? Which shares or services accept verified credentials?

Flow: enumerate -> establish context -> collect files/users/credentials -> validate access on SMB/RDP/WinRM/web/DB -> re-enumerate. Research a vulnerability only after a product/version/config match.

Access channels and file sources

These services become most useful when another finding gives you a credential, key, path or filename.

FTP - 21

Anonymous? Useful files?
Backup/config? Upload rights?

Start: ftp / Nmap NSE.

SSH - 22

Use after a key/password is found. Match the correct user to the host.

Start: ssh.

Telnet - 23

Read the banner. Test only known/default lab credentials when justified.

Start: nc / telnet.

RDP - 3389

Remote GUI access when valid Windows credentials exist.

Start: xfreerdp.

WinRM - 5985/5986

Windows shell access using verified credentials.

Start: evil-winrm / NetExec.

VNC - 5900

Check auth state and known credentials; treat it as an access channel.

Start: vncviewer.

NFS - 111 + 2049

Exports may reveal home directories, configs or SSH keys.

Start: rpcinfo / showmount.

Rsync - 873

List modules. Anonymous read may expose configs or keys.

Start: rsync.

Correlation shortcut: useful file / credential / key -> validate access -> enter session -> enumerate the new session again.

Information-rich services and databases

Small services can change the route by revealing names, users, exports, routes or credentials.

Network information

DNS 53	Records, hostnames, vhosts; zone transfer if appropriate.	Start: dig.
SNMP 161/ UDP	System, users, routes, interfaces when a community is known/weak.	Start: snmpwalk.
SMTP 25	Banner, supported commands, user behavior if applicable.	Start: nc.
RPC/NFS	Exports and files can reveal configs, keys or home data.	Start: rpcinfo / showmount.

Data services

1433 / 3306	MSSQL / MySQL: version, auth, databases with credentials.	Native client.
5432 / 6379	PostgreSQL / Redis: auth, DBs, INFO/config exposure.	psql / redis-cli.
9200	Elasticsearch: version, cluster, index/auth exposure.	curl.
27017	MongoDB: version, auth, databases.	mongosh.

Turn evidence into the next action

A credential, hash, key or version is a routing object - not the finish line.

Plaintext credential

Reuse only on services you actually discovered.

Hash

Identify type first, then crack only when it is useful to the route.

SSH key

Match the correct user, host and file permissions.

Config file

Extract hostnames, services, DB/API credentials and paths.

Tool chooser

Unknown port

Nmap - what service is here?

Known port, unknown service

Nmap -sV - what product/version is listening?

Known service

Native protocol tool - what can I enumerate without guessing?

Known product + finding

Research / SearchSploit - does a precise issue match?

Matched module

Metasploit - read module info/options, then validate target settings.

Known user + justified auth test

Hydra - only when evidence supports a small, controlled login test.

After access: enumerate users, files, interfaces and routes. If the host can see a subnet your machine cannot, consider a controlled pivot/tunnel and scan the new reachable context.